

LEAP · PIKPAK

Leap Database Plan

One company database in Azure to replace the spreadsheets: the fleet, customers, SKUs, trays, maintenance, issues and root causes, daily production, with every change tracked and backups that look after themselves.

Draft for discussion · 7 September 2026
Prepared by Chris Hamblin.

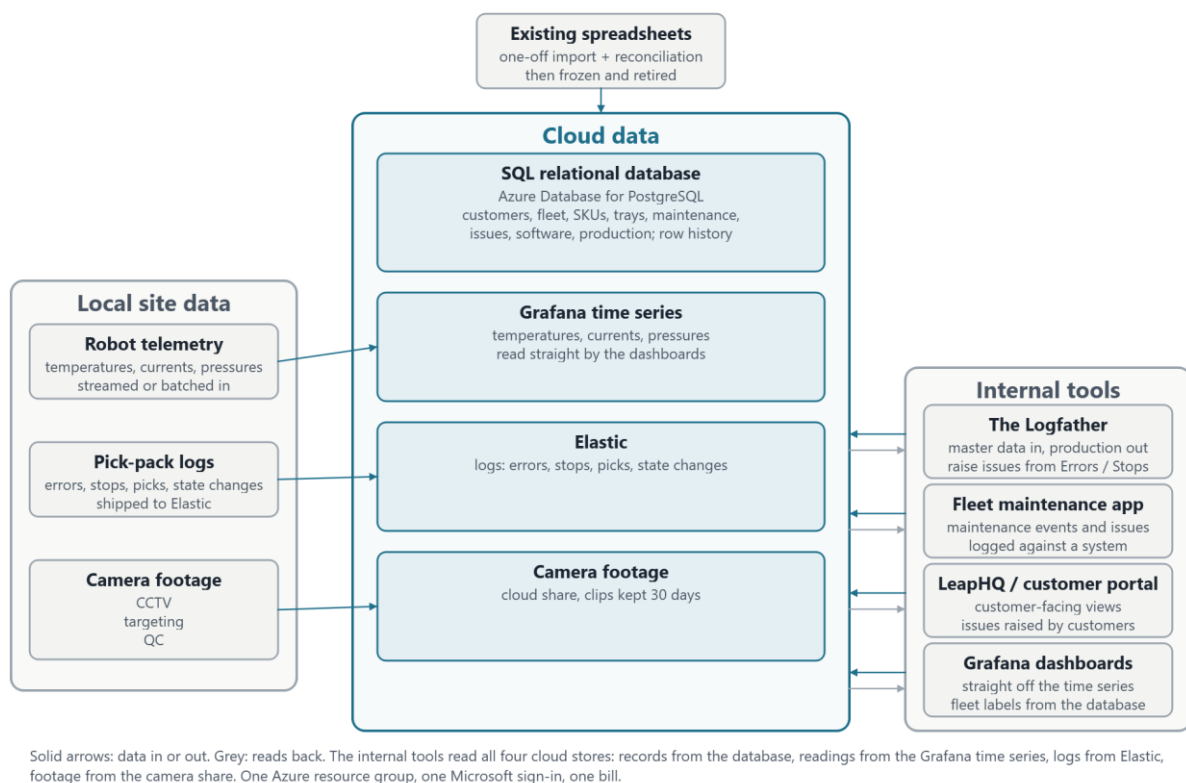
Summary

Today the fleet's information lives in spreadsheets, Teams chats, or people's heads: system serial numbers, customers, SKU numbers, tray numbers, maintenance events and root cause analyses, each in its own workbook. They are hard to join, hard to back up and impossible to audit.

This plan replaces them with one database in Azure. It is a PostgreSQL database, the same engine behind most modern web systems, run for us as a managed service so backups, patching and failover are Microsoft's job.

How the system works

- **One database, the source of truth.** Azure Database for PostgreSQL holds every record: the relational tables, JSON where a record is naturally a document, and the time series through TimescaleDB. Spreadsheets become read-only views of it, never the other way round.



Trackable

Every SQL table carries who created and last changed a row and when, and the entry app writes a history row for every change on the core tables. "What did this system's line assignment look like on 3 March" is a single query. Nothing is ever hard-deleted; rows are retired with an end date.

Easy to back up

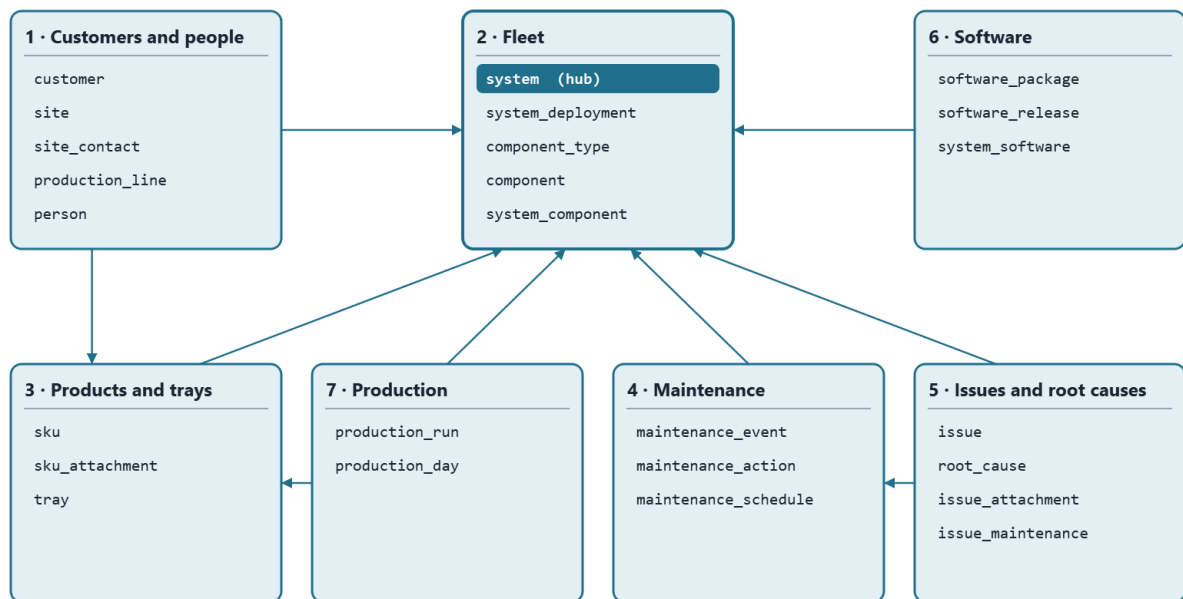
The managed service takes automatic backups with point-in-time restore for up to 35 days, optionally copied to a second region. For years-long retention a weekly dump goes to Blob Storage with a lifecycle rule, one small scheduled job. A restore drill is a ticket in the plan, not an afterthought.

The engine

Near term: Azure Database for PostgreSQL for the records; Grafana keeps the time series it holds today.

Suggested long-term: Azure Database for PostgreSQL Flexible Server, with the TimescaleDB extension, one database for everything.

The seven areas



Keys and conventions

- **Surrogate primary keys everywhere:** an integer id per table. Serial numbers, tray numbers and SKU codes are human identifiers that get retyped and corrected, so they must not be what other tables point at.
- **Natural keys as unique constraints:** a system's serial number (with the display name "PikPak 007" unique too, but only ever a label), a tray's unique number, a SKU's code, a package's git commit. Duplicates are rejected by the database.
- **Human references** on things people talk about: ISS-0001, MNT-0001, RC-0001, generated from the id and shown everywhere.
- **History as date ranges, not overwrites.** Where a system is, which component is fitted, which software runs: each is a row with from and to dates, and only one open row is allowed per subject.
- **Lookups are tables, not free text:** statuses, severities, event types, component types and issue categories, so a new value is a row, not a code change.
- **Audit on every table** plus a history table per core entity, giving the same "as of" queries a temporal table would.

Skeleton first, detail as needed

Neither a full schema agreed up front nor a table added whenever someone asks. The split is by how expensive a change is later.

- **Agree early, because it is costly to change once data and other apps depend on it:** the core entities and how they relate, the key strategy and reference scheme, the conventions above, and the engine and hosting.
- **Grow piecemeal, because it is cheap:** columns, lookup values, attachments, extra link tables and views. A column is added when someone has a row to put in it, not before. The "columns worth naming" lists below are a starting point, not a contract.

- **Keep change cheap so piecemeal is safe:** every schema change is a versioned migration, the entry screens are generated from the schema so a new column appears without bespoke work, and other apps read views rather than raw tables so a table can be reshaped without breaking them.
- **Watch for the skeleton change in disguise:** splitting one table into two, or moving a column between tables, is not a small edit. It gets a migration plan and a data check.

The tables

Key: the primary key, then the unique or natural key in brackets. Columns lists the ones worth arguing about; audit columns are on every table.

1 • Customers and people

Table	Key	Columns worth naming
customer	customer_id (name)	short_code (EVG, LWS), status, account owner, notes
site	site_id (customer_id, name)	address, timezone, notes
site_contact	site_contact_id (site_id, person_id, role)	The customer's people at a site: person, role (production manager, maintenance lead, IT, quality, commercial), is_primary, phone / email overrides, active_from, active_to. A person can hold roles at several sites.
production_line	line_id (site_id, name)	name ("Line 6"), description, shift pattern
person	person_id (email)	name, phone, organisation (own / customer / contractor), customer_id when they belong to a customer, job title, active. Used by contacts, maintenance and issues alike.

2 • Fleet

Table	Key	Columns worth naming
system	system_id (serial_number; display_name; folder_name)	The serial number "35-2300-007" is the identity; display_name "PikPak 007" is what every screen shows; folder_name "PikPak007" is the CCTV share folder, kept separate so a rename never breaks the footage path. Plus generation (Argus 1 / 2), build_date, status (in build, FAT, installed, workshop, decommissioned), notes.
system_deployment	deployment_id (one open row per system)	system, line, installed_on, removed_on, reason. Answers "where was PikPak 007 in March".
component_type	component_type_id (name)	robot, servo, drive, camera, PSU, IO card, PC, gripper, conveyor motor (anything with a serial number)
component	component_id (serial_number; part_number)	type, supplier, purchased_on, warranty_until, status
system_component	system_component_id (one open row per system + position)	system, component, position ("servo 2", "pick camera"), fitted_on, removed_on, the maintenance event that fitted it.

3 • Products and trays

No tray-type table: most trays are unique with subtle differences, so the design lives on the tray. No recipe-version table: a SKU never changes once defined and is the same for every customer that packs it.

Table	Key	Columns worth naming
tray	tray_id (unique_number)	dimensions, cavity layout, material, supplier, drawing reference; customer, site, status (in service, damaged, retired), commissioned_on, retired_on. A damaged tray is an issue with the tray linked, not a separate log.
sku	sku_id (code)	code as logged in Elastic ("EVG_picc..."), description, product dimensions and weight, products per pick, the tray it is packed in, config (JSON, the recipe), status. Not owned by a customer.
sku_attachment	attachment_id	sku, kind (photo, drawing, spec sheet), file in Blob Storage, caption, sort order, uploaded by. One SKU can carry several pictures.

4 • Maintenance

Table	Key	Columns worth naming
maintenance_event	event_id (MNT-0001)	system, type (planned service, breakdown, part replacement, software update, calibration, inspection, FAT), started_at, ended_at, downtime_minutes, performed_by, summary, external ticket
maintenance_action	action_id	event, description, component removed, component fitted, labour hours, parts cost
maintenance_schedule	schedule_id (system_id, task)	task, interval (days or picks), last done, next due. Optional in phase one.

5 • Issues and root causes

Kept deliberately simple: a root cause is an entry in a catalogue with its own number, and an issue points at one. No workflow states, no analysis records; those can be added later.

Table	Key	Columns worth naming
root_cause	root_cause_id (RC-0001; name)	The catalogue of every known root cause: name ("Vacuum solenoid stuck"), description, category (mechanical, electrical, software, operator, product, tray, environment), active. Never deleted, only retired.
issue	issue_id (ISS-0001)	system (nullable for site-level), site, tray (nullable), reported_at, reported_by, title, description, severity, root cause (nullable until known), Grafana link, Jira key, notes
issue_attachment	attachment_id	issue, kind (photo, clip, log export, document), file, caption, uploaded by
issue_maintenance	(issue_id, event_id)	links an issue to the maintenance that fixed it

6 • Software

Table	Key	Columns worth naming
software_package	package_id (name)	argus, planner, targeting, actuators, sensors, infeed, crate_change, behaviour, firmware packages
software_release	release_id (package_id, git_commit)	version, branch, released_on, approved, release notes. Where "which commits are approved for the fleet" lives.
system_software	system_software_id (one open row per system + package)	system, package, release, installed_at, removed_at, source (manual / Elastic). The Logfather already computes this from Elastic.

7 • Production

Table	Key	Columns worth naming
production_run	run_id (elastic_run_id)	system, SKU, tray, started_at, ended_at, products_packed, trays_packed, products_seen, products_rejected, pick_movements, raw counters (JSON), source. One row per run.
production_day	production_day_id (system, day, sku, tray)	The daily roll-up: products_packed, trays_packed, products_rejected, pick_movements, run_count, minutes_running, minutes_on. The table reports and Excel use.